

ALGORITHMIC EFFICIENCY & RECURSION TOOLKIT (AERT)

UNIT – 1 REPORT

1. ABSTRACT DATA TYPE (ADT)

An Abstract Data Type (ADT) is a logical description of a data structure that defines operations without specifying how they are implemented.

Example: Stack ADT supports push, pop, peek, isEmpty and size.

Stack operations take $O(1)$ time because insertion and deletion occur only at the top.

2. FACTORIAL USING RECURSION

Factorial of n is defined as:

$$n! = n \times (n-1)!$$

Base case:

$$0! = 1$$

Time Complexity: $O(n)$

Space Complexity: $O(n)$

3. FIBONACCI USING RECURSION

Naïve recursive Fibonacci:

Each call makes two more calls.

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$

Memoized Fibonacci:

Stores previously computed values.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Naïve Fibonacci becomes slow because repeated calculations occur.

4. TOWER OF HANOI

Moves required:

$$2^n - 1$$

Each step moves one disk after solving smaller subproblems.

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$

5. RECURSIVE BINARY SEARCH

Recurrence:

$$T(n) = T(n/2) + O(1)$$

Solution:

$O(\log n)$

Best Case: $O(1)$

Average Case: $O(\log n)$

Worst Case: $O(\log n)$

6. ALGORITHM PARADIGMS

Binary Search → Divide and Conquer (splits array into halves)

Memoized Fibonacci → Dynamic Programming (stores subproblem results)

Tower of Hanoi → Recursion (solves smaller instances)

Real-life examples:

Divide & Conquer – Searching a word in dictionary

Dynamic Programming – Route optimization

Greedy – Selecting highest value coin first

CONCLUSION:

Recursive algorithms simplify complex problems and efficiency improves using optimization techniques like memoization.